Ex.No : 6	Exercises on MVC Architecture
Date :	

AIM :

To study and implement the Model-View-Controller (MVC) architecture by developing simple web applications that separate data, user interface, and control logic.

1. Question 1 - Online Shopping Cart System

Problem Statement:

Design a simple Online Shopping Cart System using the MVC (Model-View-Controller) architecture. The webpage should display a list of products with their prices. Users should be able to add products to the cart, remove products, view the total price, and proceed to checkout.

Given:

- A list of products with their names, prices, and available stock.
- An Add to Cart button and a Remove button.
- HTML, CSS, and JavaScript using the MVC Design Pattern.
- The Model manages product and cart data.
- The View displays products, cart items, and total price.
- The Controller handles user actions such as adding, removing, and checking out.

What to do:

- Create a webpage to display the available products.
- Display the product name, price, and Add to Cart button.
- Add the selected product to the shopping cart.
- Remove products from the cart using the Remove button.
- Display the total number of items and the total price.
- Check product availability before adding to the cart.
- Click the Checkout button to complete the purchase.
- Display a success message after checkout.
- Show an error message if the selected product is out of stock.

Proposed Solution:

Use the MVC (Model-View-Controller) architecture to separate the application into three components:

- Model: Stores and manages product details, cart items, prices, quantities, and stock availability.
- View: Displays the product list, shopping cart, total amount, and checkout status to the user.
- Controller: Receives user actions (Add, Remove, Checkout), updates the Model, and refreshes the View accordingly.

Program:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<title>Inventory Management System</title>
<link rel="stylesheet" href="style.css" />
</head>
<body>

 <header>
  <h1>Inventory Management System</h1>
  <nav>
   <button class="tab-btn active" data-tab="inventory">Inventory</button>
   <button class="tab-btn" data-tab="sales">Process Sale</button>
   <button class="tab-btn" data-tab="reports">Reports</button>
  </nav>
 </header>

 <main>

  <!-- INVENTORY TAB -->
  <section id="inventory-tab" class="tab-content active">
   <h2>Add New Product</h2>
   <form id="add-product-form">
    <input type="text" id="product-name" placeholder="Product name" required />
    <input type="text" id="product-category" placeholder="Category" required />
    <input type="number" id="product-price" placeholder="Price" min="0" step="0.01" required />
    <input type="number" id="product-quantity" placeholder="Initial quantity" min="0" required />
    <button type="submit">Add Product</button>
   </form>

   <h2>Current Inventory</h2>
   <table id="inventory-table">
    <thead>
     <tr>
      <th>ID</th><th>Name</th><th>Category</th><th>Price</th><th>Qty</th><th>Status</th><
th>Actions</th>
     </tr>
    </thead>
    <tbody><!-- rows injected by view.js --></tbody>
   </table>
```

```
  </section>

  <!-- SALES TAB -->
  <section id="sales-tab" class="tab-content">
    <h2>Process a Sale</h2>
    <form id="sale-form">
      <select id="sale-product"></select>
      <input type="number" id="sale-quantity" placeholder="Quantity" min="1" value="1" required />
      <button type="submit">Sell</button>
    </form>

    <h2>Recent Transactions</h2>
    <table id="sales-table">
      <thead>
        <tr><th>Date</th><th>Product</th><th>Qty</th><th>Total</th></tr>
      </thead>
      <tbody></tbody>
    </table>
  </section>

  <!-- REPORTS TAB -->
  <section id="reports-tab" class="tab-content">
    <h2>Inventory & Sales Reports</h2>
    <div id="report-summary" class="cards"></div>

    <h3>Top Selling Products</h3>
    <ol id="top-products"></ol>

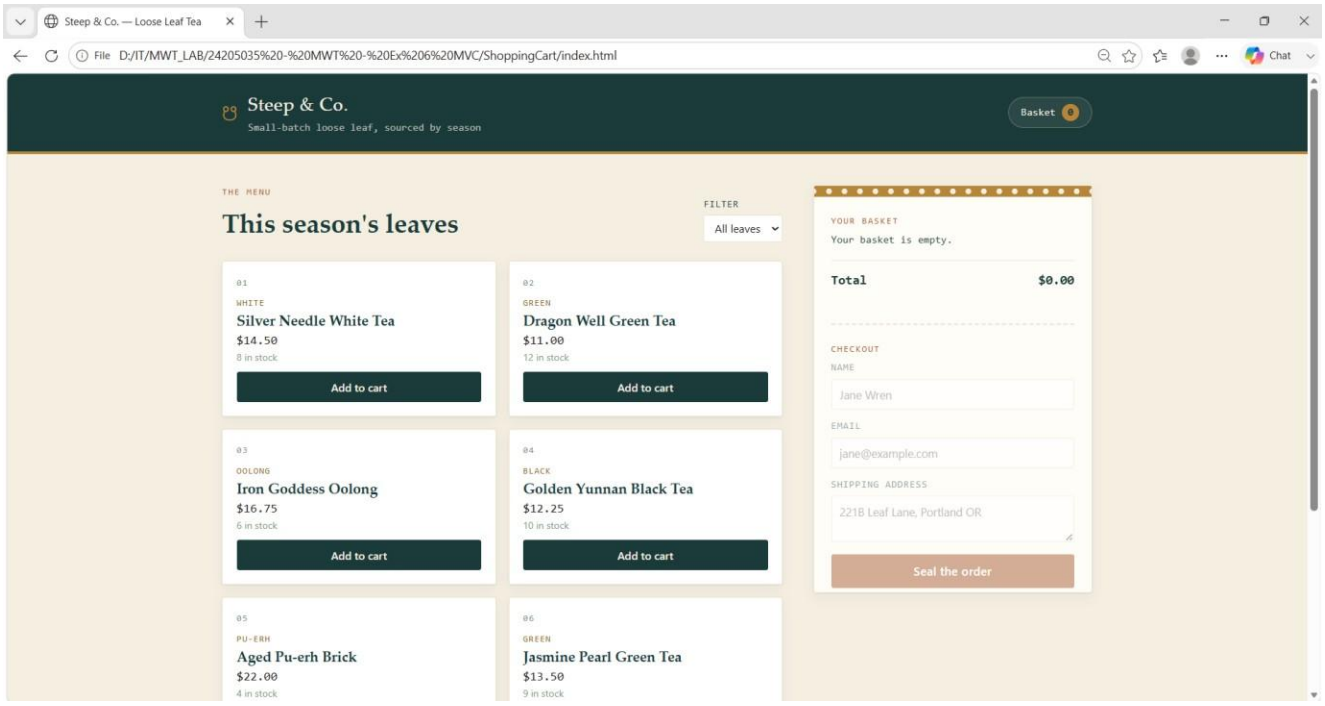
    <h3>Low Stock Alerts (&lt; 5 units)</h3>
    <ul id="low-stock-list"></ul>
  </section>

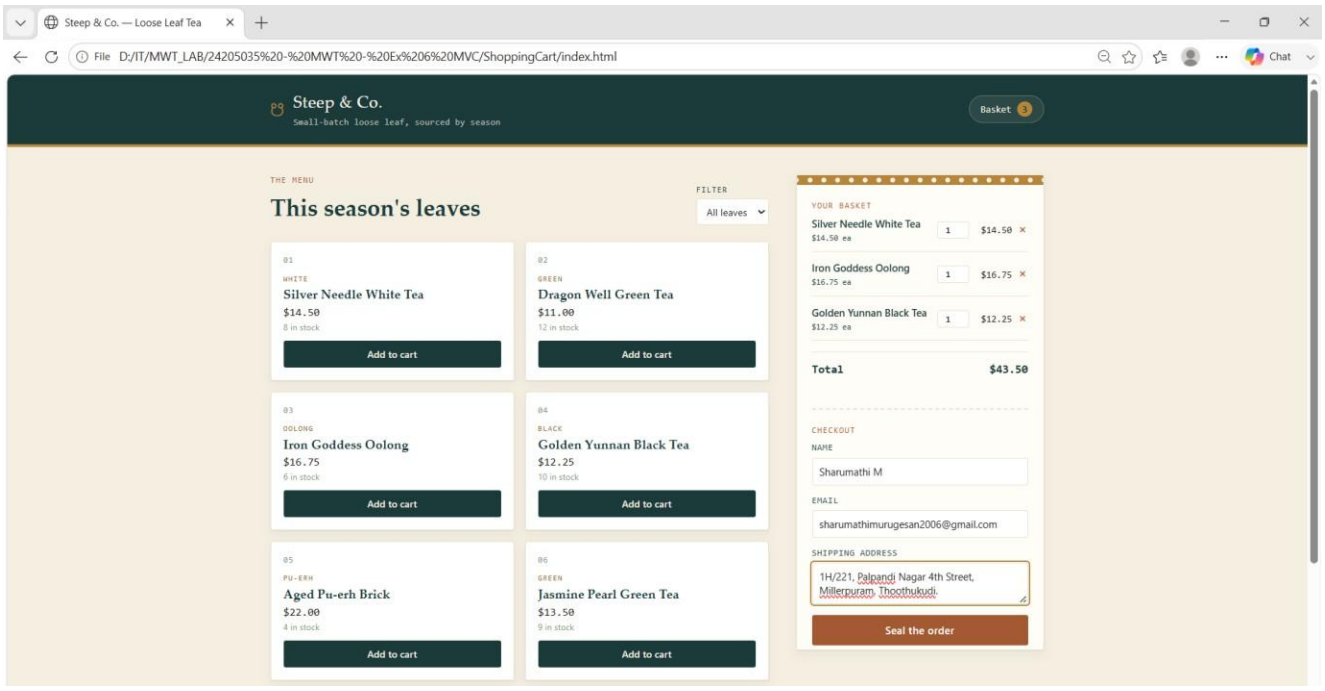
</main>

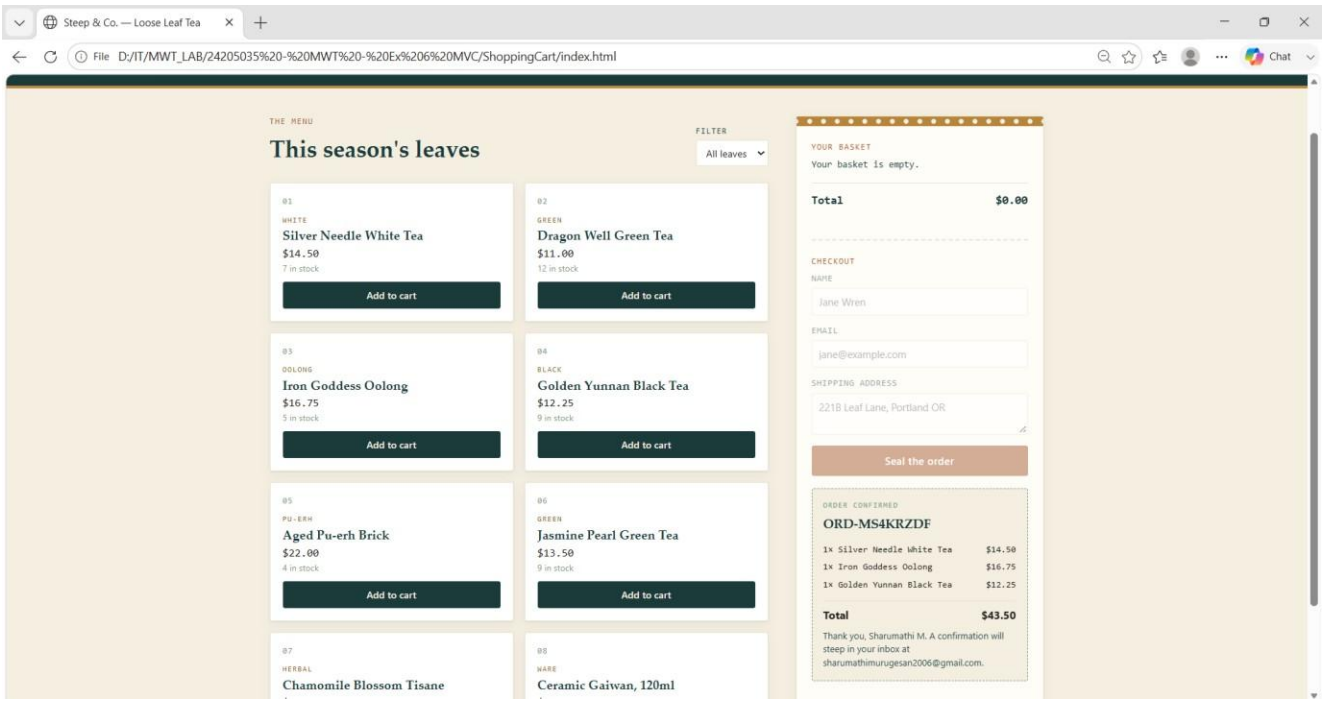
<div id="toast"></div>

<script src="model.js"></script>
<script src="view.js"></script>
<script src="controller.js"></script>
<script src="app.js"></script>
</body>
</html>
```

Output:


Steep & Co. — Loose Leaf Tea
File D:/IT/MWT_LAB/24205035%20-%20MWT%20-%20Ex%206%20MVC/ShoppingCart/index.html
Chat
Steep & Co.
Small-batch loose leaf, sourced by season
Basket 0
THE MENU
This season's leaves
FILTER
All leaves
01
WHITE
Silver Needle White Tea
$14.50
8 in stock
Add to cart
02
GREEN
Dragon Well Green Tea
$11.00
12 in stock
Add to cart
03
OOLONG
Iron Goddess Oolong
$16.75
6 in stock
Add to cart
04
BLACK
Golden Yunnan Black Tea
$12.25
10 in stock
Add to cart
05
PU-ERH
Aged Pu-erh Brick
$22.00
4 in stock
06
GREEN
Jasmine Pearl Green Tea
$13.50
9 in stock
YOUR BASKET
Your basket is empty.
Total $0.00
CHECKOUT
NAME
Jane Wren
EMAIL
jane@example.com
SHIPPING ADDRESS
221B Leaf Lane, Portland OR
Seal the order


Steep & Co. — Loose Leaf Tea
File D:/IT/MWT_LAB/24205035%20-%20MWT%20-%20Ex%206%20MVC/ShoppingCart/index.html
Chat
Steep & Co.
Small-batch loose leaf, sourced by season
Basket 3
THE MENU
This season's leaves
FILTER
All leaves
Silver Needle White Tea
$14.50
Add to cart
Dragon Well Green Tea
$11.00
Add to cart
Iron Goddess Oolong
$16.75
Add to cart
Golden Yunnan Black Tea
$12.25
Add to cart
Aged Pu-erh Brick
$22.00
Add to cart
Jasmine Pearl Green Tea
$13.50
Add to cart
YOUR BASKET
Silver Needle White Tea $14.50 ea 1 $14.50
Iron Goddess Oolong $16.75 ea 1 $16.75
Golden Yunnan Black Tea $12.25 ea 1 $12.25
Total $43.50
CHECKOUT
NAME
Sharumathi M
EMAIL
sharumathimurugesan2006@gmail.com
SHIPPING ADDRESS
1H/221, Palpandi Nagar 4th Street, Millerpuram, Thoothukudi.
Seal the order



2. Question 4 - User Authenication System

Problem Statement:

Design a simple User Authentication System with a Login and Registration page. The system should allow users to create an account using a username and password, log in with valid credentials, and display a welcome message after successful authentication. Invalid login attempts should display an appropriate error message.

Given:

- Username and password entered by the user.
- Register and Login buttons.
- HTML, CSS, and JavaScript using the MVC (Model-View-Controller) architecture.
- The Model stores user credentials and validates login information.
- The View displays the registration form, login form, and authentication messages.
- The Controller handles user registration, login, logout, and validation requests.

What to do:

- Create a Registration form with username and password fields.
- Create a Login form with username and password fields.
- Register a new user by storing the entered credentials.
- Validate the username and password during login.
- Display a Welcome message if the login is successful.
- Show an error message for invalid username or password.
- Prevent duplicate usernames during registration.

- Allow the user to Logout and return to the login page.
- Display appropriate success and error messages.

Proposed Solution:

Use the MVC (Model-View-Controller) architecture to implement the User Authentication System.

- Model: Stores user credentials, validates usernames and passwords, and manages user data.
- View: Displays the registration page, login page, welcome page, and authentication messages.
- Controller: Handles user actions such as registration, login, logout, input validation, and updates the View based on the Model's response.

Program:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Account — Simple Auth Demo</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>

  <header class="site-header">
    <div class="wrap header-inner">
      <p class="brand-name">Account</p>
      <div class="nav-right">
        <span id="nav-user" class="nav-user" hidden></span>
        <button id="nav-logout" class="btn btn-ghost" hidden>Log out</button>
      </div>
    </div>
  </header>

  <main class="wrap">

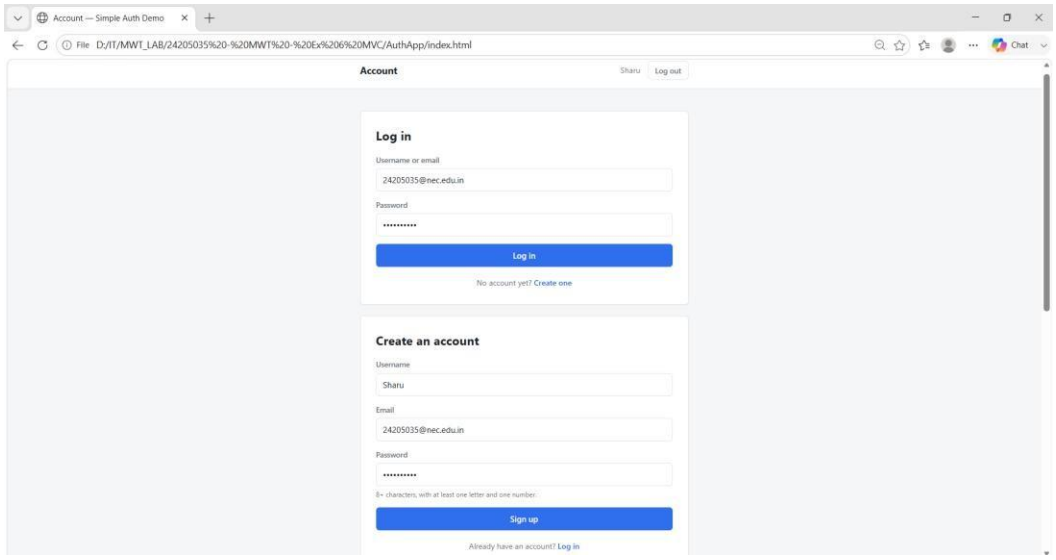
    <!-- ============ Login screen ============ -->
    <section id="screen-login" class="screen">
      <div class="card">
        <h1>Log in</h1>
        <form id="login-form" class="auth-form" novalidate>
          <p id="login-message" class="form-message" hidden></p>
          <label>
            Username or email
            <input type="text" name="identifier" autocomplete="username" required />
```

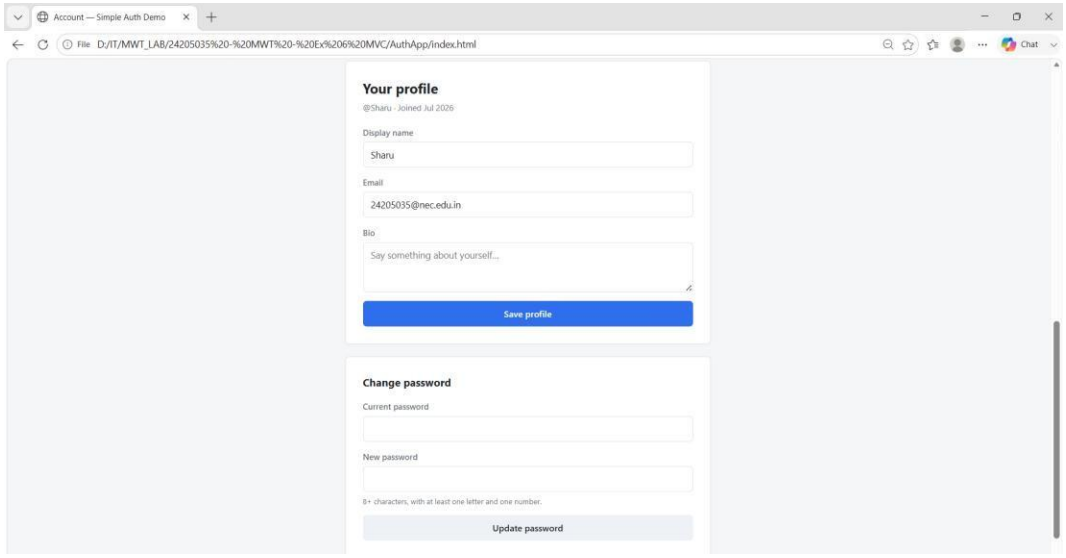
```
        </label>
        <label>
          Password
          <input type="password" name="password" autocomplete="current-password" required />
        </label>
        <button type="submit" class="btn btn-primary">Log in</button>
      </form>
      <p class="switch-line">No account yet? <a href="#" id="go-to-register">Create one</a></p>
    </div>
  </section>
  <!-- ============ Registration screen ============ -->
  <section id="screen-register" class="screen" hidden>
    <div class="card">
      <h1>Create an account</h1>
      <form id="register-form" class="auth-form" novalidate>
        <p id="register-message" class="form-message" hidden></p>
        <label>
          Username
          <input type="text" name="username" autocomplete="username" required />
        </label>
        <label>
          Email
          <input type="email" name="email" autocomplete="email" required />
        </label>
        <label>
          Password
          <input type="password" name="password" autocomplete="new-password" required />
          <span class="hint">8+ characters, with at least one letter and one number.</span>
        </label>
        <button type="submit" class="btn btn-primary">Sign up</button>
      </form>
      <p class="switch-line">Already have an account? <a href="#" id="go-to-login">Log in</a></p>
    </div>
  </section>
  <!-- ============ Profile screen ============ -->
  <section id="screen-profile" class="screen" hidden>
    <div class="card">
      <h1>Your profile</h1>
      <p class="profile-meta"><span id="profile-username"></span> &middot; <span id="profile-
joined"></span></p>
      <form id="profile-form" class="auth-form" novalidate>
        <p id="profile-message" class="form-message" hidden></p>
```

```
        <label>
          Display name
          <input type="text" name="displayName" required />
        </label>
        <label>
          Email
          <input type="email" name="email" required />
        </label>
        <label>
          Bio
          <textarea name="bio" rows="3" placeholder="Say something about
yourself&hellip;"></textarea>
        </label>
        <button type="submit" class="btn btn-primary">Save profile</button>
      </form>
    </div>

    <div class="card">
      <h2>Change password</h2>
      <form id="password-form" class="auth-form" novalidate>
        <p id="password-message" class="form-message" hidden></p>
        <label>
          Current password
          <input type="password" name="currentPassword" autocomplete="current-password" required
/>
        </label>
        <label>
          New password
          <input type="password" name="newPassword" autocomplete="new-password" required />
          <span class="hint">8+ characters, with at least one letter and one number.</span>
        </label>
        <button type="submit" class="btn btn-secondary">Update password</button>
      </form>
    </div>
  </section>
 </main>
 <script src="model.js"></script>
 <script src="view.js"></script>
 <script src="controller.js"></script>
 <script src="app.js"></script>
</body>
</html>
```

Output:


Account — Simple Auth Demo
File D:/IT/MWT_LAB/24205035%20-%20MWT%20-%20Ex%206%20MVC/AuthApp/index.html
Chat
Account
Sharu
Log out
Log in
Username or email
24205035@nec.edu.in
Password
Log in
No account yet? Create one
Create an account
Username
Sharu
Email
24205035@nec.edu.in
Password
8+ characters, with at least one letter and one number.
Sign up
Already have an account? Log in


Account — Simple Auth Demo
File D:/IT/MWT_LAB/24205035%20-%20MWT%20-%20Ex%206%20MVC/AuthApp/index.html
Chat
Your profile
@Sharu · Joined Jul 2026
Display name
Sharu
Email
24205035@nec.edu.in
Bio
Say something about yourself...
Save profile
Change password
Current password
New password
8+ characters, with at least one letter and one number.
Update password

3. Question 10 - Inventory Management System

Problem Statement:

Design a simple Inventory Management System using the MVC (Model-View-Controller) architecture. The webpage should display a list of products with their details. The user should be able to add new products, update product quantity, delete products, and view the available stock.

Given:

- Product details entered by the user (Product ID, Product Name, Quantity, and Price).
- Buttons for Add Product, Update Product, Delete Product, and Search Product.
- HTML, CSS, and JavaScript using the MVC Design Pattern.
- The Model stores and manages inventory data.
- The View displays the inventory information.
- The Controller processes user actions and updates the Model and View.

What to do:

- Create a webpage with input fields for Product ID, Product Name, Quantity, and Price.
- Add a new product to the inventory.
- Search for a product using its Product ID or Product Name.
- Update the quantity or price of an existing product.
- Delete a product from the inventory.
- Display the complete inventory in a table.
- Show the total number of available products.
- Display a success message after each operation.
- Show an error message if the product is not found or if invalid data is entered.

Proposed Solution:

Use the MVC (Model-View-Controller) architecture to organize the Inventory Management System into three components:

- Model: Stores and manages product information such as Product ID, Name, Quantity, and Price.
- View: Displays the inventory list, product details, and operation status to the user through the webpage.
- Controller: Handles user actions such as adding, updating, searching, and deleting products by interacting with the Model and updating the View.

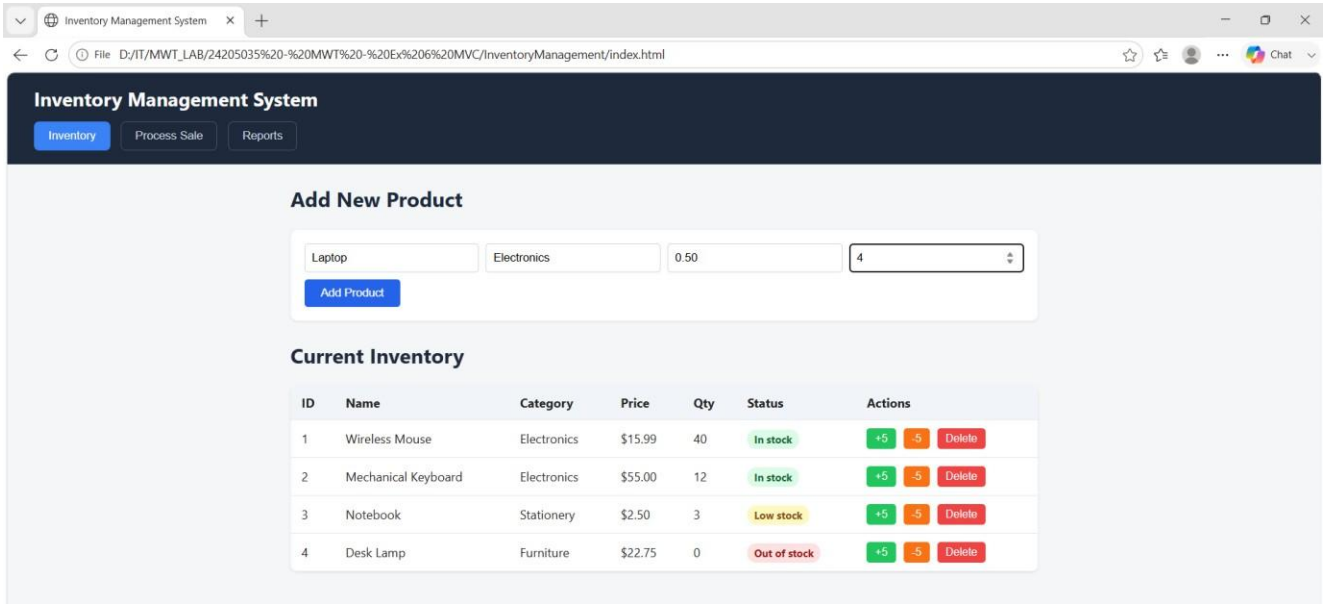
Program:

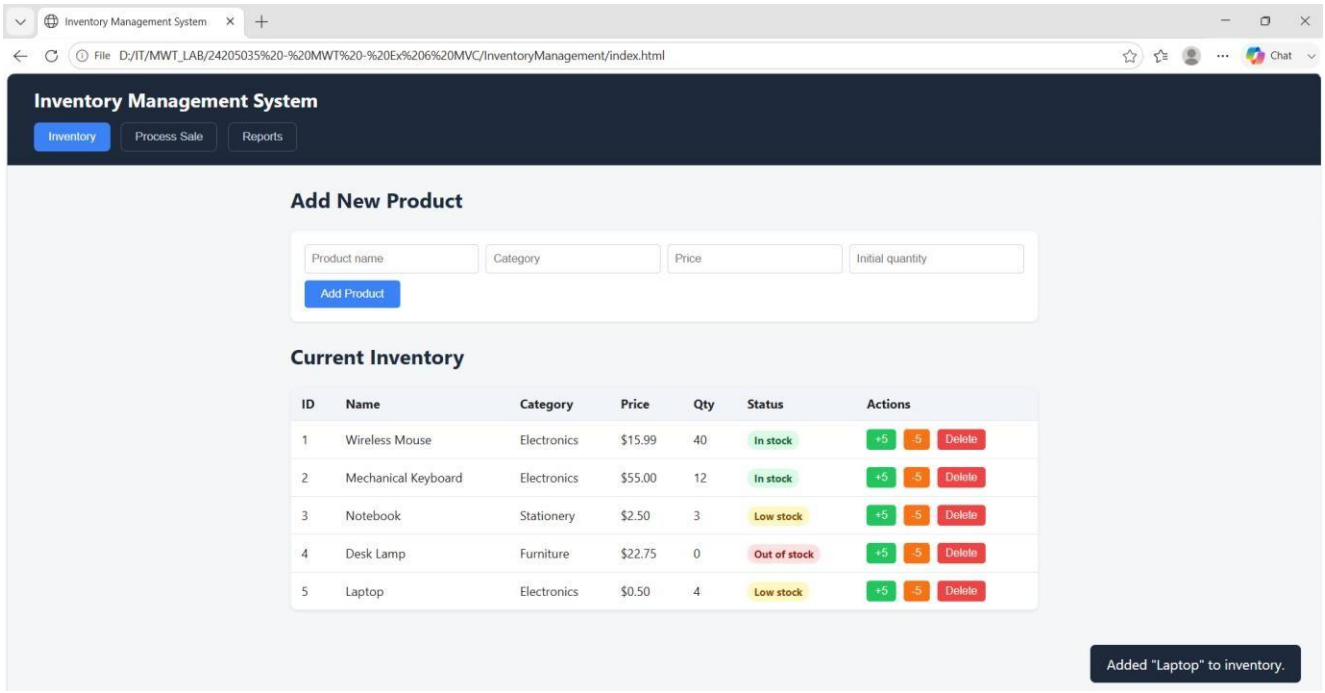
```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<title>Inventory Management System</title>
<link rel="stylesheet" href="style.css" />
```

```
</head>
<body>
  <header>
    <h1>Inventory Management System</h1>
    <nav>
      <button class="tab-btn active" data-tab="inventory">Inventory</button>
      <button class="tab-btn" data-tab="sales">Process Sale</button>
      <button class="tab-btn" data-tab="reports">Reports</button>
    </nav>
  </header>
  <main>
    <!-- INVENTORY TAB -->
    <section id="inventory-tab" class="tab-content active">
      <h2>Add New Product</h2>
      <form id="add-product-form">
        <input type="text" id="product-name" placeholder="Product name" required />
        <input type="text" id="product-category" placeholder="Category" required />
        <input type="number" id="product-price" placeholder="Price" min="0" step="0.01" required />
        <input type="number" id="product-quantity" placeholder="Initial quantity" min="0" required />
        <button type="submit">Add Product</button>
      </form>
      <h2>Current Inventory</h2>
      <table id="inventory-table">
        <thead>
          <tr>
            <th>ID</th><th>Name</th><th>Category</th><th>Price</th><th>Qty</th><th>Status</th><
th>Actions</th>
          </tr>
        </thead>
        <tbody><!-- rows injected by view.js --></tbody>
      </table>
    </section>
    <!-- SALES TAB -->
    <section id="sales-tab" class="tab-content">
      <h2>Process a Sale</h2>
      <form id="sale-form">
        <select id="sale-product"></select>
        <input type="number" id="sale-quantity" placeholder="Quantity" min="1" value="1" required />
        <button type="submit">Sell</button>
      </form>
      <h2>Recent Transactions</h2>
      <table id="sales-table">
```

```
      <thead>
        <tr><th>Date</th><th>Product</th><th>Qty</th><th>Total</th></tr>
      </thead>
      <tbody></tbody>
    </table>
  </section>
  <!-- REPORTS TAB -->
  <section id="reports-tab" class="tab-content">
    <h2>Inventory & Sales Reports</h2>
    <div id="report-summary" class="cards"></div>
    <h3>Top Selling Products</h3>
    <ol id="top-products"></ol>
    <h3>Low Stock Alerts (&lt; 5 units)</h3>
    <ul id="low-stock-list"></ul>
  </section>
 </main>
 <div id="toast"></div>
 <script src="model.js"></script>
 <script src="view.js"></script>
 <script src="controller.js"></script>
 <script src="app.js"></script>
</body>
</html>
```

Output:





Problem Understanding & Approach	Implementation & Execution	Tool / Technology Usage	Code Quality & Documentation	Viva	Time Management	Total
(10)	(15)	(10)	(5)	(5)	(5)	(50)

RESULT :

Thus, the applications were successfully implemented using the MVC Architecture. The Model, View, and Controller components worked together to perform the required operations efficiently, demonstrating the principles of modularity, maintainability, and code reusability. Thus, the objectives of the experiment were successfully achieved.